

PPGEEC2327 – Tópicos Especiais em Processamento Inteligente da Informação

Inteligência artificial aplicada à geofísica

Capítulo 3 — Métodos especializados de descida do gradiente

Baseado em Schuster, G.T. — *Machine Learning Methods in Geoscience*

Professor: Tiago Barros

Programa de Pós-graduação em Engenharia Elétrica e de Computação (PPgEEC) — UFRN

Sumário da Aula

Motivação

Método do Momentum

Adagrad

Adadelta e RMSprop

Nesterov Accelerated Gradient

Adam

AdaMax e Estratégia Híbrida

Busca Linear Numérica

Gradiente Conjugado (CG)

Síntese e Fechamento

Motivação

Do Capítulo 2 para o Capítulo 3

Recapitulando

O Cap. 2 introduziu a descida do gradiente iterativa como forma acessível de resolver grandes sistemas de equações ($\mathbf{X}\mathbf{w} = \mathbf{t}$).

Agora, introduzimos métodos mais sofisticados:

- Métodos **ad hoc** desenvolvidos pela comunidade de ML (Momentum, Adagrad, Adadelta, RMSprop, Adam, AdaMax).
- O método do **Gradiente Conjugado** (CG).

Ideia-Chave

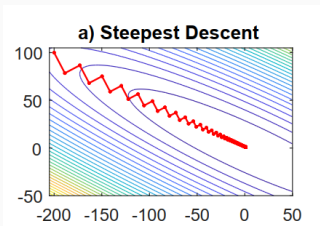
Característica comum: usam informação de gradientes de **iterações anteriores** (e, às vezes, de uma iteração futura) para estimar melhor o próximo passo — como um esquiador escolhendo a melhor trilha morro abaixo.

Do capítulo 2 para o capítulo 3



O Problema: Vales Estreitos e Contornos Distorcidos

- Funções objetivo com contornos alongados em certas direções → sintoma de problema **mal-posto**.
- Elipses com grande razão de aspecto → convergência **lenta** da descida mais íngreme (*steepest descent*).
- A descida mais íngreme **oscila** entre subir e descer as paredes íngremes do vale.



Definição

O gradiente pode ser decomposto em $\nabla \epsilon^{(k)} = \nabla \epsilon_{\parallel}^{(k)} + \nabla \epsilon_{\perp}^{(k)}$: componente ao longo do fundo do vale ($\parallel$) e componente subindo as paredes ($\perp$).

Método do Momentum

Método do Momentum

Se dois gradientes sucessivos têm componentes perpendiculares quase opostas:

$$\nabla \epsilon^{(k)} + \nabla \epsilon^{(k-1)} \approx 2 \nabla \epsilon_{\parallel}^{(k)}$$

A soma de dois gradientes sucessivos reforça a direção ao longo do fundo do vale e cancela as oscilações.

Fórmula do momentum (Sutton, 1986):

$$\mathbf{v}^{(k)} = \beta \mathbf{v}^{(k-1)} + \alpha \nabla \epsilon^{(k)} \qquad \mathbf{w}^{(k+1)} = \mathbf{w}^{(k)} - \mathbf{v}^{(k)}$$

- $\beta \approx 0.9$ (recomendado); α ajustado por tentativa e erro.
- Convergência tipicamente mais rápida que SD quando as elipses têm razão de aspecto grande.

Momentum vs. Newton: Função de Rosenbrock

$$\epsilon = 100(w_2 - w_1^2)^2 + (1 - w_1)^2, \quad \text{mínimo em } (1, 1)$$

- Passo fixo pequeno ($\alpha = 0.0045$): pouco progresso em 10 iterações.
- Passo maior: iterados **divergem** para longe do mínimo.
- **Newton** converge em 4 iterações — usa informação de curvatura (Hessiana) a cada passo.

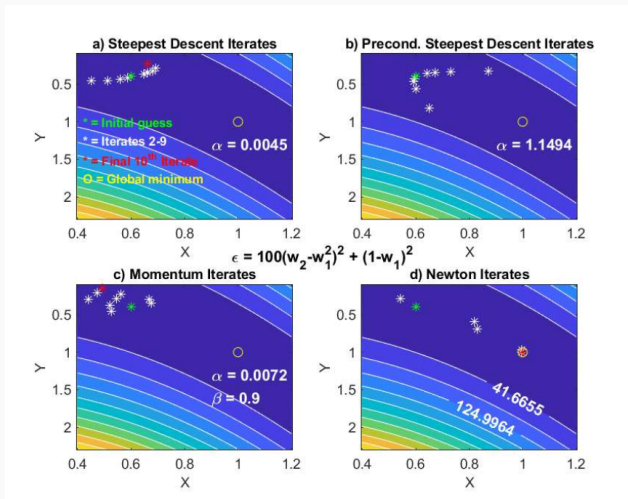
Por que Newton vence?

Métodos de 1ª ordem (SD, momentum) usam passo fixo sem informação de curvatura; a curvatura de funções altamente não lineares muda de ponto a ponto.

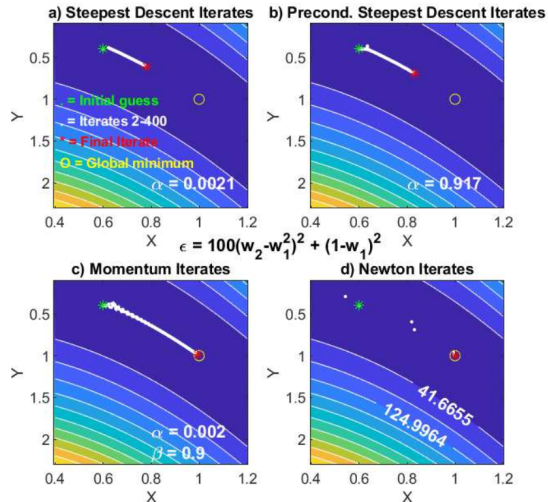
Ideia-Chave

Estratégia de *backtracking* por bisseção: se ϵ não decresce, reduzir $\alpha := \alpha/2$ e testar novamente.

Momentum vs. Newton: Função de Rosenbrock



Momentum vs. Newton: Função de Rosenbrock

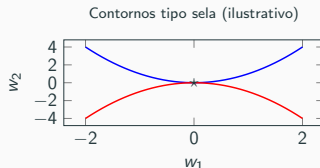


Caso Ponto de Sela: Cuidado com Suposições

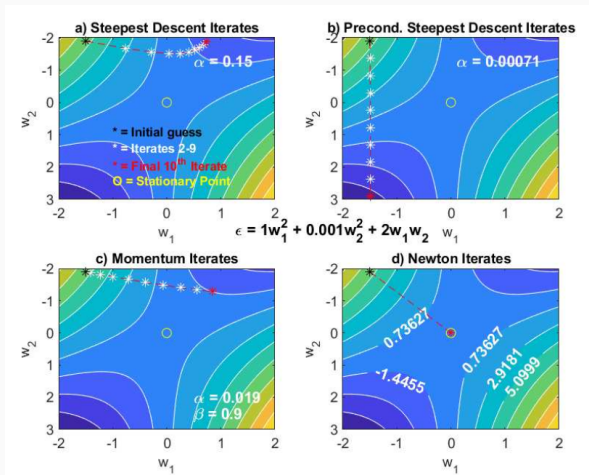
Para $\epsilon = w_1^2 + 0.001w_2^2 + w_1w_2$, a Hessiana é

$$\mathbf{H} = \begin{bmatrix} 2 & 1 \\ 1 & 0.002 \end{bmatrix}, \quad \lambda_1 = -0.41, \lambda_2 = 2.41 \Rightarrow \mathbf{H} \text{ não é SPD (ponto de sela)}$$

- Apenas o método de Newton (2ª ordem) encontra o ponto estacionário em 1 iteração.
- Métodos de 1ª ordem (SD, momentum, pré-condicionada) supõem $\mathbf{H}$ diagonalmente dominante $\rightarrow$ suposição **falha** aqui.
- Todos os métodos de 1ª ordem **falham** em atingir o ponto de sela.



Caso Ponto de Sela: Cuidado com Suposições

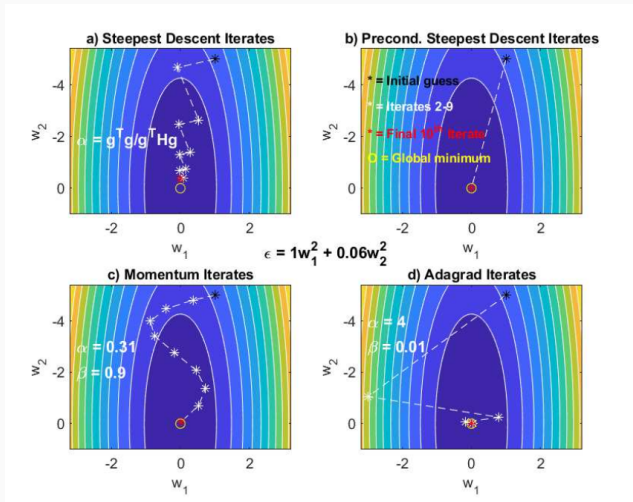


Quadrática SPD Amigável: Comparando os Métodos

$$\epsilon = w_1^2 + 0.06w_2^2, \quad \mathbf{H} = \begin{bmatrix} 2 & 0 \\ 0 & 0.12 \end{bmatrix} \quad (\text{razão de aspecto} > 7:1)$$

- **SD pré-condicionada:** converge em **1** iteração (pré-condicionador = $\mathbf{H}^{-1}$ exato).
- **SD** (passo exato): ~ 10 iterações.
- **Momentum:** passo fixo $\alpha = 0.54$, ~ 10 iterações.
- **Adagrad:** ~ 4 iterações.

Quadrática SPD Amigável: Comparando os Métodos



Adagrad

Adagrad: Passo Adaptativo por Componente

Diferente de SD/momentum (mesmo α para todas as componentes), o **Adagrad** (Duchi et al., 2011) usa passo individual por parâmetro:

$$w_i^{(k+1)} = w_i^{(k)} - \frac{\alpha}{\underbrace{\sqrt{\sum_{k'=1}^k (g_i^{(k')})^2 + \eta}}_{\text{pré-condicionador}}} \frac{\partial \epsilon^{(k)}}{\partial w_i}$$

- $\eta \approx 10^{-8}$ (Ruder, 2017): fator de amortecimento.
- Componente $g_{\parallel}$ (fundo do vale, pequena magnitude) $\rightarrow$ passo **amplificado**.
- Componente $g_{\perp}$ (parede íngreme, grande magnitude) $\rightarrow$ passo **reduzido**.

Ideia-Chave

Adagrad elimina a necessidade de ajuste manual de α ao longo das iterações — basta $\alpha = 0.01$.

Limitação

Com muitas iterações, o denominador cresce demais $\rightarrow$ convergência **estagna**.

Adadelata e RMSprop

Adadelta: Corrigindo o Denominador Crescente

Restringir a soma a uma janela de κ termos (média móvel exponencial):

$$\tilde{E}((g_i^{(k)})^2) = \gamma \sum_{k'=k-\kappa}^{k-1} (g_i^{(k')})^2 + (1 - \gamma)(g_i^{(k)})^2, \quad \gamma = 0.9$$

$$w_i^{(k+1)} = w_i^{(k)} - \frac{\alpha}{\sqrt{\tilde{E}((g_i^{(k)})^2) + \eta}} \frac{\partial \epsilon^{(k)}}{\partial w_i} \quad (\text{igual ao } \mathbf{RMSprop} \text{ de Hinton, } \gamma = 0.9, \alpha = 0.001)$$

Correção de unidades — Adadelta completo:

$$w_i^{(k+1)} = w_i^{(k)} - \frac{\sqrt{\tilde{E}((\Delta w_i^{(k-1)})^2) + \eta}}{\sqrt{\tilde{E}((g_i^{(k)})^2) + \eta}} \frac{\partial \epsilon^{(k)}}{\partial w_i}$$

- Numerador com unidades de $\Delta w_i \Rightarrow$ **não requer** ajuste manual de α .
- Comportamento: oscila nas paredes íngremes, similar à SD, até convergir.

Nesterov Accelerated Gradient

Nesterov Accelerated Gradient (NAG)

Substituir o gradiente atual pelo gradiente “espiado à frente” (*peek-ahead*):

$$\mathbf{v}^{(k)} = \beta \mathbf{v}^{(k-1)} + \alpha \nabla \epsilon(\mathbf{w}^{(k)} - \beta \mathbf{v}^{(k-1)})$$

$$\mathbf{w}^{(k+1)} = \mathbf{w}^{(k)} - \mathbf{v}^{(k)}$$

- Estimativa mais precisa da inclinação entre $\mathbf{w}^{(k-1)}$ e $\mathbf{w}^{(k+1)}$.
- Direções de atualização mais produtivas; evita máximos locais.
- Bengio et al. (2013): convergência mais rápida que momentum em **redes recorrentes** (Cap. 12).

Adam

Adam: Estimativa Adaptativa de Momentos

Kingma e Ba (2014) — combina momentum com passo adaptativo por componente. **Primeiro momento** (média ponderada dos gradientes):

$$m_i^{(k)} = \beta_1 m_i^{(k-1)} + (1 - \beta_1) g_i^{(k)}$$

Segundo momento (média dos gradientes ao quadrado):

$$\sigma_i^{2(k)} = \beta_2 \sigma_i^{2(k-1)} + (1 - \beta_2) (g_i^{(k)})^2$$

Correção de viés (pois $m_i^{(0)} = \sigma_i^{2(0)} = 0$):

$$\hat{m}_i^{(k)} = \frac{m_i^{(k)}}{1 - \beta_1^k}, \quad \hat{\sigma}_i^{2(k)} = \frac{\sigma_i^{2(k)}}{1 - \beta_2^k}$$

Fórmula de Atualização do Adam

$$w_i^{(k+1)} = w_i^{(k)} - \frac{\alpha}{\sqrt{\hat{\sigma}_i^{2(k)} + \eta}} \hat{m}_i^{(k)}$$

Hiperparâmetros típicos: $\beta_1 = 0.9$, $\beta_2 = 0.99$, $\eta = 10^{-8}$.

Com regularização L_2 (Bushaev, 2018):

$$w_i^{(k+1)} = w_i^{(k)} - \frac{\alpha}{\sqrt{\hat{\sigma}_i^{2(k)} + \eta}} \hat{m}_i^{(k)} - \alpha \lambda w_i^{(k)}$$

Ideia-Chave

Adam = combinação de **RMSprop** + **SGD com momentum**. Considerado por Bushaev (2018) um dos melhores otimizadores para aprendizado profundo, com adoção crescente.

- Trajetória de convergência tipicamente **mais suave** que Adadelta.

Adam VS Adadelata

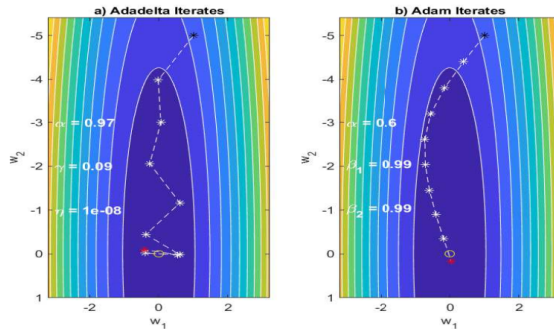


Figure 3.6: Results for finding the minimizer of $\epsilon = w_1^2 + 0.06w_2^2$ by the a) Adadelata (equation 3.10) and b) Adam methods. Here the number of terms in the summation of equation 3.9 is 4.

Código MATLAB (fragmento): Passo do Adam

```
1  alpha=.6; beta1=.9; beta2=.99; lam=10^-8;  
2  mx = beta1*mx + (1-beta1)*gx;  
3  my = beta1*my + (1-beta1)*gy;  
4  sx = beta2*sx + (1-beta2)*gx^2;  
5  sy = beta2*sy + (1-beta2)*gy^2;  
6  mxh = mx/(1-beta1^it);    myh = my/(1-beta1^it);  
7  sxh = sx/(1-beta2^it);    syh = sy/(1-beta2^it);  
8  g(1) = alpha*mxh/(sqrt(sxh)+lam);  
9  g(2) = alpha*myh/(sqrt(syh)+lam);
```

Equivalente Keras/TensorFlow: `tf.keras.optimizers.Adam(...)`

AdaMax e Estratégia Híbrida

Substitui a média do primeiro momento pela norma L_∞ :

$$u_i^{(k)} = \max(\beta_2 m_i^{(k-1)}, |g_i^{(k)}|)$$

$$w_i^{(k+1)} = w_i^{(k)} - \frac{\alpha}{u_i^{(k)} + \eta} \hat{m}_i^{(k)}$$

- Valores padrão: $\alpha = 0.02$, $\eta = 0.002$, $\beta_1 = 0.9$, $\beta_2 = 0.99$.
- Ignora componentes pequenas do gradiente, favorecendo a de maior magnitude.
- Norma L_∞ : mais estável para problemas inversos (Ruder, 2017).

Estratégia Híbrida de Comprimento de Passo

- Keskar e Socher (2017): **Adam** supera SGD no início do treinamento, mas o desempenho **estagna** depois.
- **Recomendação**: usar Adam nas iterações iniciais, depois trocar para **SGD** quando a convergência do Adam estagnar.

Ideia-Chave

Adam aplica passo **não uniforme** por parâmetro; SGD aplica o **mesmo** passo a todos.
Wilson et al. (2017): escalonamento não uniforme pode **inibir a generalização**.

Busca Linear Numérica

Busca Linear Numérica (*Numerical Line Search*)

Quando o funcional é altamente não linear, a busca linear **exata** pode falhar. **Busca linear quadrática**: avaliar ϵ em pontos ao longo da direção de descida $\Delta \mathbf{w}$ e interpolar:

$$\epsilon(\mathbf{w} + \alpha \Delta \mathbf{w}) = \frac{(\alpha - \alpha_1)(\alpha - \alpha_2)}{\alpha_1 \alpha_2} \epsilon(\mathbf{w}) + \frac{\alpha(\alpha - \alpha_2)}{\alpha_1(\alpha_1 - \alpha_2)} \epsilon(\mathbf{w} + \alpha_1 \Delta \mathbf{w}) + \frac{\alpha(\alpha - \alpha_1)}{\alpha_2(\alpha_2 - \alpha_1)} \epsilon(\mathbf{w} + \alpha_2 \Delta \mathbf{w})$$

Variante com derivada direcional $\epsilon(\mathbf{w})'$ (evita uma avaliação extra):

$$\epsilon(\mathbf{w} + \alpha \Delta \mathbf{w}) = \frac{\alpha^2}{\alpha_1^2} [\epsilon(\mathbf{w} + \alpha_1 \Delta \mathbf{w}) - \alpha_1 \epsilon(\mathbf{w})' - \epsilon(\mathbf{w})] + \alpha \epsilon(\mathbf{w})' + \epsilon(\mathbf{w})$$

- Minimizar em relação a α : derivar, igualar a zero, resolver.
- Alternativa prática: *backtracking* por bisseção (seção 4.3.3).

Gradiente Conjugado (CG)

Gradiente Conjugado — Motivação

Sistema SPD: $\mathbf{H}\mathbf{w} = \mathbf{b}$, com objetivo

$$\epsilon = \frac{1}{2}\mathbf{w}^T \mathbf{H}\mathbf{w} - \mathbf{w}^T \mathbf{b}$$

- Gradiente: $\nabla\epsilon(\mathbf{w}) = \mathbf{H}\mathbf{w} - \mathbf{b}$; no ótimo, $\mathbf{H}\mathbf{w}^* = \mathbf{b}$.
- Ao contrário da SD (que oscila), o CG usa direções **conjugadas**:

$$(\mathbf{x}, \mathbf{H}\mathbf{y}) = 0 \Rightarrow \mathbf{x}, \mathbf{y} \text{ são conjugados}$$

- Para um sistema 2×2 : CG converge em **2** iterações. Em geral, converge em no máximo N iterações para um sistema $N \times N$ SPD.

Gradiente Conjugado

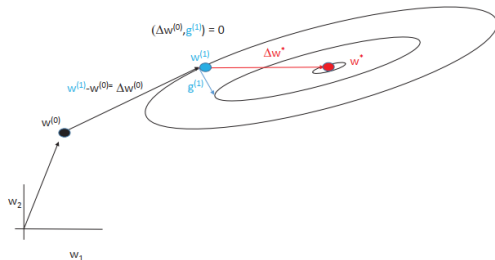


Figure 3.7: Iterative solutions $\mathbf{w}^{(k)}$ showing that $\Delta \mathbf{w}^{(0)} \perp \mathbf{g}^{(1)}$ and that $\Delta \mathbf{w}^{(0)}$ is conjugate to $\Delta \mathbf{w}^*$ because $\Delta \mathbf{w}^{(0)} \perp \mathbf{g}^{(1)} \rightarrow (\Delta \mathbf{w}^{(0)}, \mathbf{H}\mathbf{w}^{(1)} - \mathbf{b}) = (\Delta \mathbf{w}^{(0)}, \mathbf{H}(\mathbf{w}^{(1)} - \mathbf{w}^*)) = (\Delta \mathbf{w}^{(0)}, \mathbf{H}\Delta \mathbf{w}^*) = 0$.

Direção de Atualização Conjugada

Perpendicularidade: $\Delta \mathbf{w}^{(0)} \perp \mathbf{g}^{(1)}$, ou seja $(\Delta \mathbf{w}^{(0)}, \mathbf{H}\mathbf{w}^{(1)} - \mathbf{b}) = 0$. Substituindo $\mathbf{b} = \mathbf{H}\mathbf{w}^*$:

$$(\Delta \mathbf{w}^{(0)}, \mathbf{H}(\mathbf{w}^* - \mathbf{w}^{(1)})) = 0 \quad \Rightarrow \quad \Delta \mathbf{w}^{(0)} \text{ conjugado a } \Delta \mathbf{w}^*$$

Nova direção de busca:

$$\Delta \mathbf{w}^* = -\mathbf{g}^{(1)} + \beta \Delta \mathbf{w}^{(0)} \quad \beta = \frac{-(\Delta \mathbf{w}^{(0)}, \mathbf{H}\mathbf{g}^{(1)})}{(\Delta \mathbf{w}^{(0)}, \mathbf{H}\Delta \mathbf{w}^{(0)})}$$

Generalizando para $N \times N$:

$$\beta = \frac{(\Delta \mathbf{w}^{(k-1)}, \mathbf{H}\mathbf{g}^{(k)})}{(\Delta \mathbf{w}^{(k-1)}, \mathbf{H}\Delta \mathbf{w}^{(k-1)})}$$

1. Chute inicial $\mathbf{w}^{(0)}$; direção inicial $\Delta \mathbf{w}^{(0)} = -\mathbf{g}^{(0)}$. $k = 0$.
2. Atualizar $\mathbf{w}^{(k+1)} = \mathbf{w}^{(k)} + \alpha \Delta \mathbf{w}^{(k)}$, com α dado pelo passo exato (Eq. 2.57).
3. Calcular novo gradiente $\mathbf{g}^{(k+1)} = \nabla \epsilon(\mathbf{w}^{(k+1)})$ e nova direção

$$\Delta \mathbf{w}^{(k+1)} = -\mathbf{g}^{(k+1)} + \beta \Delta \mathbf{w}^{(k)}$$

usando a fórmula de **Polak-Ribière**:

$$\beta = \frac{(\mathbf{g}^{(k+1)} - \mathbf{g}^{(k)}, \mathbf{g}^{(k+1)})}{(\mathbf{g}^{(k)}, \mathbf{g}^{(k)})}$$

4. $k := k + 1$; repetir até convergência.

Para funcionais não lineares, β “reseta” a direção para SD quando o progresso estagna (conjugacidade se perde).

Código MATLAB: Gradiente Conjugado (Sistema Quadrado)

```
1 L = [400 1; 1 4]; t = [5 2]';
2 xk = zeros(2,1);
3 gk = (L*xk - t); dk = -gk;
4 for q = 1:3
5     alpha = -dk'*gk/(dk'*L*dk);
6     xk1 = xk + alpha*dk;
7     gk1 = gk + alpha*L*dk;
8     beta = gk1'*(gk1-gk)/(gk'*gk);    % Polak-Ribiere
9     dk1 = -gk1 + beta*dk;
10    dk = dk1; gk = gk1; xk = xk1;
11 end
```

Métodos Quasi-Newton (QN)

- Usam gradiente e atualizações de iterações passadas para aproximar a **inversa da Hessiana**.
- Chamados de métodos de 2ª **ordem** (vs. 1ª ordem como SD).
- Custo: $O(m) \times$ mais caro que métodos de 1ª ordem, onde m = número de histórias armazenadas.

Resultado empírico surpreendente

Berahas et al. (2019): para uma rede neural e o conjunto MNIST, o Adam (1ª **ordem**) superou significativamente os métodos QN testados em velocidade de convergência.

Síntese e Fechamento

Resumo do Capítulo

Nove métodos de descida do gradiente descritos e testados: SD, SD pré-condicionada, Newton, momentum, Adadelta, Adagrad, Adam, AdaMax e CG.

- **Newton**: melhor desempenho geral (usa gradiente e curvatura), mas custo $O(N^3)$ proibitivo para problemas de ML realistas ($N > 10^6$).
- Métodos com **pré-condicionador** por parâmetro (SD, Adam, Adagrad, Adadelta) — próximos melhores; aproximam a inversa da Hessiana e arredondam as elipses de ϵ .
- **CG**: única classe que impõe **conjugacidade** entre direções sucessivas.
- **Adam**: popularidade crescente; combinação de RMSprop + momentum; pode ter generalização inferior → estratégia híbrida com SGD.

Lembrete crucial

Nenhum método de gradiente garante convergência ao mínimo global em problemas altamente não lineares — evitar mínimos locais continua sendo um grande desafio.

Obrigado!

Perguntas e discussão